

1. Краткий отчёт по TDD (Lab 5, вариант 21)

Тема: разработка класса «Четырёхугольник на плоскости» и тестов JUnit с применением TDD (Test Driven Development).

1.1. Цель

Разработать:

- класс `Point2D` (точка на плоскости);
- класс `Quadrilateral` (четырёхугольник на плоскости) с методами вычисления **площади и периметра**;
- определение типа четырёхугольника (квадрат / прямоугольник / ромб / произвольный);
- обработку ошибочных ситуаций через исключения;
- JUnit-тесты и демонстрационное приложение.

1.2. Методика TDD

Работа выполнялась итерациями **Red → Green → Refactor**: 1) **Red**: пишется новый тест, который падает. 2) **Green**: добавляется минимальный код, чтобы тест прошёл. 3) **Refactor**: упрощение и улучшение кода без изменения поведения, тесты остаются зелёными.

1.3. Последовательность разработки в стиле TDD (по тестам)

Тесты находятся в `src/test/java/by/bsu/tiokrs/lab5/geometry/QuadrilateralTest.java`.

Важно: по TDD **каждое** требование сначала формулировалось в виде падающего теста (**Red**), затем добавлялся минимальный код (**Green**), после чего выполнялось упрощение (**Refactor**) при зелёных тестах. Для надёжности тестов сделано **больше, чем публичных методов**: тесты разбиты на маленькие проверки поведения и граничных случаев (это нормальная практика в TDD).

Ниже перечислены все тесты и что именно они «закрепляют» (какой контракт поведения фиксируют).

1.3.1. 1) `squareAreaAndPerimeter()`

Red: сначала проверка `perimeter()` для квадрата (8), затем `area()` (4) и `type()` (SQUARE).

Green: реализованы `Point2D`, `Quadrilateral`, `perimeter()`, `area()` (shoelace), базовая логика `type()` для квадрата.

Refactor: вынесено расстояние в `Point2D.distanceTo()`, введены допуски для `double`.

1.3.2. 2) `squareRotatedStillSquare()`

Red: квадрат после поворота должен оставаться квадратом.

Green: `type()` и вычисления не должны зависеть от ориентации; добавлены устойчивые проверки через скалярное произведение (прямые углы) и равенство длин (с `epsilon`).

1.3.3. 3) `rectangleDetectedButNotSquare()`

Red: прямоугольник 3×2 должен распознаваться как RECTANGLE.

Green: уточнена логика `type()` (правые углы + параллельность противоположных сторон), чтобы отличать прямоугольник от квадрата.

1.3.4. 4) rectangleRotatedStillRectangle()

Red: прямоугольник после поворота должен оставаться прямоугольником.

Green: закреплена инвариантность `type()` к поворотам и численная устойчивость.

1.3.5. 5) rhombusDetectedButNotRectangle()

Red: ромб должен распознаваться как RHOMBUS.

Green: добавлена проверка ромба как параллелограмма с равными сторонами.

1.3.6. 6) rhombusRotatedStillRhombus()

Red: ромб после поворота должен оставаться ромбом.

Green: закреплена инвариантность распознавания ромба к поворотам.

1.3.7. 7) arbitraryQuadrilateral()

Red: произвольный четырёхугольник должен классифицироваться как ARBITRARY.

Green: в `type()` добавлен корректный “fallback” на ARBITRARY.

1.3.8. 8) translationDoesNotChangeAreaOrPerimeterOrType()

Red: перенос фигуры не должен менять площадь, периметр и тип.

Green: закреплено требование инвариантности к переносу (это тест на корректность формул и отсутствие зависимостей от абсолютного положения).

1.3.9. 9) reversingVertexOrderKeepsAreaAndPerimeter()

Red: обход вершин по часовой/против часовой стрелки — это та же фигура.

Green: `area()` использует `abs(...)`, а `type()` не должен ломаться при смене ориентации обхода.

1.3.10. 10) concaveQuadrilateralIsArbitrary()

Red: вогнутый простой четырёхугольник не должен считаться квадратом/прямоугольником/ромбом.

Green: добавлена проверка выпуклости: специальные типы определяются только для выпуклых.

1.3.11. 11) parallelogramButNotRectangleOrRhombusIsArbitrary()

Red: параллелограмм, который не прямоугольник и не ромб, должен быть ARBITRARY.

Green: логика `type()` уточнена: отдельные типы выделяются только при выполнении их условий (иначе ARBITRARY).

1.3.12. 12) kiteIsArbitrary()

Red: «дельтоид» (kite) не должен ошибочно распознаваться как ромб/прямоугольник.

Green: закреплено отсутствие ложной классификации при неподходящих условиях параллельности/углов.

1.3.13. 13) rejectsNonFinitePointCoordinates()

Red: координаты точки не могут быть NaN/Infinity.

Green: в Point2D добавлена проверка Double.isFinite(...) и выброс IllegalArgumentException.

1.3.14. 14) rejectsNullVertices()

Red: null вершины — ошибка.

Green: в конструкторе Quadrilateral добавлены Objects.requireNonNull(...) (ожидается NullPointerException).

1.3.15. 15) rejectsDuplicateVertices()

Red: совпадающие вершины — ошибка.

Green: добавлена проверка различия вершин и выброс IllegalArgumentException.

1.3.16. 16) rejectsSelfIntersectingQuadrilateral()

Red: самопересечение (“бантик”) — ошибка.

Green: добавлена проверка пересечения непересекающихся сторон (simple polygon), выброс IllegalArgumentException.

1.3.17. 17) rejectsCrossedOrderOfRectangleVertices()

Red: даже если точки образуют прямоугольник как множество, «перепутанный» порядок вершин создаёт самопересечение и должен быть отклонён.

Green: та же проверка простоты (self-intersection) в конструкторе фиксируется отдельным тестом.

1.3.18. 18) rejectsZeroArea()

Red: нулевая площадь (все точки коллинеарны) — ошибка.

Green: добавлена проверка area() > 0 в конструктор и выброс IllegalArgumentException.

1.4. Демонстрационное приложение

Файл DemoApp:

- создаёт массив случайных объектов Quadrilateral разных типов;
- считает количество по типам;
- для каждой группы находит min/max по площади и по периметру и печатает в консоль.

1.5. Итог

В результате получены:

- рабочая реализация Point2D и Quadrilateral с обработкой ошибок через исключения;
- набор JUnit 5 тестов, написанных итеративно по TDD;
- консольное демо-приложение.